

UMLytics – Fully Dressed Use Cases

AI-Powered UML Generation and Design Intelligence Tool

Group Members:

Ammar Bin Omer (24I-0500) | Haris Zahid (24I-0643) | Shahmeer Jadoon (24I-0879)

Table of Contents

1. Create and Initialize Project
2. Generate UML Diagram from Natural Language
3. Generate UML Diagram from Source Code
4. Refine UML Diagram Design
5. Evaluate UML Design Quality
6. Consult AI for Design Guidance
7. Generate Class Structure Suggestions
8. Analyze Uploaded UML Diagram
9. Maintain Project Data
10. Retrieve Existing Project
11. Provide Voice-Based Input
12. Export UML Diagram for External Use

Use Case 1: Create and Initialize Project

Use Case Name	Create and Initialize Project
Scope	UMLytics System
Level	User-Goal
Primary Actor	Software Designer
Stakeholders & Interests	• Software Designer: Wants a structured workspace to organize UML diagrams and projects. • Database System: Must ensure data integrity and persistent storage of project metadata.
Preconditions	• The UMLytics application is installed and running. • The database connection is active.
Postconditions	• A new project record is created in the database. • The project workspace is initialized and ready for diagram creation.
Main Success Scenario	
Actor Action	System Response
1. The Software Designer selects "New Project" from the main menu.	
	2. The system displays the project creation form with fields for name, description, and diagram type.
3. The Software Designer enters the project name and an optional description.	
	4. The system validates the input (non-empty name, unique project name).
5. The Software Designer confirms the creation.	
	6. The system creates the project record in the database and initializes the workspace.
7. The Software Designer views the newly created project dashboard.	
	8. The system displays the empty project workspace with available tools and options.

Extensions

3a. The Software Designer enters a duplicate or empty project name:

3a1. The system displays a validation error message.

3a2. The Software Designer corrects the input and re-submits.

6a. Database connection fails during project creation:

6a1. The system displays an error notification.

6a2. The system rolls back any partial changes.

6a3. The Software Designer retries after connection is restored.

Use Case 2: Generate UML Diagram from Natural Language

Use Case Name	Generate UML Diagram from Natural Language
Scope	UMLytics System
Level	User-Goal
Primary Actor	Software Designer
Stakeholders & Interests	• Software Designer: Wants automated, accurate UML diagram generation from textual descriptions to speed up the design process. • AI Engine: Processes natural language input and returns structured UML data.
Preconditions	• An active project exists and is open. • The AI Engine service is available and connected.
Postconditions	• A valid UML class diagram is generated and rendered in the workspace. • The generated diagram is associated with the current project.
Main Success Scenario	
Actor Action	System Response
1. The Software Designer navigates to the diagram generation panel within the project.	
	2. The system displays the natural language input area and generation controls.
3. The Software Designer enters a textual description of the desired system (e.g., "A library system with Book, Member, and Loan classes").	
	4. The system captures and validates the input for minimum length and coherence.
5. The Software Designer clicks "Generate Diagram".	
	6. The system sends the description to the AI Engine via the LLM API.
7. The Software Designer waits while a progress indicator is shown.	
	8. The AI Engine processes the input and returns structured UML class data (classes, attributes, methods, relationships).

9. The Software Designer reviews the rendered diagram on the canvas.	
	10. The system parses the AI response, constructs the internal UML model, and renders the diagram.
Extensions	4a. The input is too short or unintelligible: 4a1. The system prompts the Software Designer to provide a more detailed description. 6a. The AI Engine service is unavailable: 6a1. The system displays a connectivity error. 6a2. The Software Designer retries later or checks network settings. 8a. The AI returns an invalid or unparseable response: 8a1. The system notifies the user of the error. 8a2. The system offers to retry with a refined prompt.

Use Case 3: Generate UML Diagram from Source Code

Use Case Name	Generate UML Diagram from Source Code
Scope	UMLytics System
Level	User-Goal
Primary Actor	Software Designer
Stakeholders & Interests	• Software Designer: Wants to reverse-engineer UML diagrams from existing codebases to understand or document system design. • System: Must correctly parse source code and extract structural information.
Preconditions	• An active project exists and is open. • The source code file(s) are in a supported language (e.g., Java).
Postconditions	• A UML class diagram reflecting the source code structure is generated and displayed. • Extracted classes, attributes, methods, and relationships are stored in the internal model.
Main Success Scenario	
Actor Action	System Response
1. The Software Designer selects "Generate from Code" in the project workspace.	
	2. The system displays a file upload dialog with supported format information.
3. The Software Designer uploads one or more source code files.	
	4. The system validates the file format and checks for parsing compatibility.
5. The Software Designer confirms the upload.	
	6. The system invokes the code parser (e.g., JavaParser) to extract classes, attributes, methods, and relationships.
7. The Software Designer views the generated diagram.	
	8. The system constructs the UML model and renders the class diagram on the canvas.

Extensions

4a. The uploaded file is in an unsupported format:

4a1. The system displays an error indicating supported formats.

4a2. The Software Designer uploads a compatible file.

6a. The parser encounters syntax errors in the source code:

6a1. The system displays a parsing error report with line numbers.

6a2. The Software Designer fixes the code and re-uploads.

Use Case 4: Refine UML Diagram Design

Use Case Name	Refine UML Diagram Design
Scope	UMLytics System
Level	User-Goal
Primary Actor	Software Designer
Stakeholders & Interests	• Software Designer: Wants to iteratively improve diagram designs by modifying classes, attributes, and relationships.
Preconditions	• A UML diagram exists in the current project workspace. • The diagram editor is accessible.
Postconditions	• The UML diagram reflects all modifications made by the Software Designer. • The updated diagram model is stored in the project.
Main Success Scenario	
Actor Action	System Response
1. The Software Designer selects a diagram from the project.	
	2. The system loads the diagram into the interactive editor canvas.
3. The Software Designer modifies classes (adds/removes/renames classes, attributes, or methods).	
	4. The system updates the internal UML model in real-time and reflects changes on the canvas.
5. The Software Designer adds, removes, or changes relationships between classes.	
	6. The system validates the structural integrity of the diagram (e.g., no orphan relationships).
7. The Software Designer confirms and saves the refined design.	
	8. The system persists the updated diagram model to the database.

Extensions

4a. The Software Designer enters an invalid attribute type:

4a1. The system highlights the error and suggests valid types.

6a. A relationship creates a circular dependency:

6a1. The system warns about the design issue.

6a2. The Software Designer decides whether to proceed or correct.

8a. Database save fails:

8a1. The system notifies the user and retains in-memory changes.

Use Case 5: Evaluate UML Design Quality

Use Case Name	Evaluate UML Design Quality
Scope	UMLytics System
Level	User-Goal
Primary Actor	Software Designer
Stakeholders & Interests	- Software Designer: Wants AI-driven feedback on design quality based on SDA principles to improve the design. - AI Engine: Evaluates the diagram against coupling, cohesion, and SOLID principles.
Preconditions	- A UML diagram exists in the current project. - The AI Engine service is available.
Postconditions	- A structured design evaluation report is generated and displayed. - The evaluation results are stored in the project history.
Main Success Scenario	
Actor Action	System Response
1. The Software Designer selects a diagram and clicks "Evaluate Design".	
	2. The system serializes the diagram and sends it to the AI Engine with evaluation criteria (coupling, cohesion, SOLID).
3. The Software Designer waits while the evaluation runs.	
	4. The AI Engine analyzes the design and returns structured feedback with scores and suggestions.
5. The Software Designer reviews the evaluation report.	
	6. The system displays the feedback in a categorized panel (e.g., Coupling: Low, SRP: Violation in Class X).

Extensions

2a. The diagram is empty or has fewer than two classes:

2a1. The system notifies the user that evaluation requires a minimum diagram complexity.

4a. The AI Engine fails to respond:

4a1. The system shows an error and offers a retry option.

4b. The AI returns ambiguous feedback:

4b1. The system presents the raw feedback with a disclaimer about interpretation.

Use Case 6: Consult AI for Design Guidance

Use Case Name	Consult AI for Design Guidance
Scope	UMLytics System
Level	User-Goal
Primary Actor	Software Designer
Stakeholders & Interests	• Software Designer: Wants contextual AI guidance on UML design decisions and SDA principles through a conversational interface. • AI Engine: Provides conversational responses grounded in design principles.
Preconditions	• The chat interface is available. • The AI Engine service is connected.
Postconditions	• The AI has responded to the design query. • The conversation is stored in chat history for the project.
Main Success Scenario	
Actor Action	System Response
1. The Software Designer opens the AI chat panel.	
	2. The system displays the chat interface with conversation history (if any).
3. The Software Designer types a design question (e.g., "Should I use inheritance or composition for Vehicle and Engine?").	
	4. The system sends the query along with project context to the AI Engine.
5. The Software Designer waits for the response.	
	6. The AI Engine generates a contextual response explaining trade-offs and recommendations.
7. The Software Designer reads and optionally follows up with another question.	
	8. The system displays the response and appends it to the conversation history.

Extensions

4a. The AI Engine is unavailable:

4a1. The system displays a connection error.

4a2. The Software Designer retries when the service is restored.

6a. The AI response is irrelevant or off-topic:

6a1. The Software Designer rephrases the question.

6a2. The system re-sends the refined query.

Use Case 7: Generate Class Structure Suggestions

Use Case Name	Generate Class Structure Suggestions
Scope	UMLytics System
Level	User-Goal
Primary Actor	Software Designer
Stakeholders & Interests	• Software Designer: Wants AI-generated class skeletons and code structure suggestions to jumpstart implementation from the diagram. • AI Engine: Generates code structure suggestions based on the diagram model.
Preconditions	• A UML class diagram exists in the current project. • The AI Engine service is available.
Postconditions	• Class skeleton code or structure suggestions are generated and displayed. • The suggestions are stored in the project for reference.
Main Success Scenario	
Actor Action	System Response
1. The Software Designer selects a diagram and clicks "Generate Structure Suggestions".	
	2. The system extracts the class model (classes, attributes, methods, relationships) and sends it to the AI Engine.
3. The Software Designer waits while the AI processes the request.	
	4. The AI Engine generates class skeletons or structural code suggestions based on the diagram.
5. The Software Designer reviews the generated suggestions.	
	6. The system displays the code suggestions in a formatted code viewer with syntax highlighting.
7. The Software Designer optionally copies or exports the generated code.	
	8. The system provides copy and export functionality for the suggestions.

Extensions

2a. The diagram has no classes defined:

2a1. The system notifies the user that at least one class is required.

4a. AI Engine failure:

4a1. The system shows an error and offers retry.

4a2. The Software Designer retries after service recovery.

Use Case 8: Analyze Uploaded UML Diagram

Use Case Name	Analyze Uploaded UML Diagram
Scope	UMLytics System
Level	User-Goal
Primary Actor	Software Designer
Stakeholders & Interests	• Software Designer: Wants to upload an existing UML diagram image for AI-assisted interpretation and analysis. • AI Engine: Interprets the uploaded diagram image and extracts design elements.
Preconditions	• A project is open. • The image file is in a supported format (PNG, JPG, etc.). • The AI Engine service is available.
Postconditions	• The uploaded diagram is analyzed and results are displayed. • Extracted design elements are optionally imported into the project model.
Main Success Scenario	
Actor Action	System Response
1. The Software Designer selects "Upload UML Image" from the project tools.	
	2. The system displays a file chooser dialog for image upload.
3. The Software Designer selects and uploads the UML diagram image.	.
	4. The system validates the image format and file size
5. The Software Designer confirms the upload for analysis.	
	6. The system sends the image to the AI Engine for interpretation.
7. The Software Designer reviews the extracted design elements and analysis.	

	8. The system displays the AI interpretation (identified classes, relationships, observations) and offers to import elements into the project model.
Extensions	4a. The image format is unsupported or the file is corrupted: 4a1. The system displays an error with supported format information. 4a2. The Software Designer selects a valid image. 6a. AI Engine cannot interpret the image: 6a1. The system notifies the user that the image quality is insufficient. 6a2. The Software Designer uploads a clearer image.

Use Case 9: Maintain Project Data

Use Case Name	Maintain Project Data
Scope	UMLytics System
Level	User-Goal
Primary Actor	Software Designer
Stakeholders & Interests	• Software Designer: Wants to update, organize, and manage project metadata and settings. • Database System: Must maintain data consistency and apply updates reliably.
Preconditions	• At least one project exists in the system. • The Software Designer has the project open.
Postconditions	• Project metadata is updated in the database. • All associated diagrams and history remain intact.
Main Success Scenario	
Actor Action	System Response
1. The Software Designer navigates to project settings.	.
	2. The system displays the current project details (name, description, creation date, last modified).
3. The Software Designer modifies the project name, description, or associated settings.	.
	4. The system validates the updated information
5. The Software Designer confirms the changes.	
	6. The system updates the project record in the database and refreshes the display.
7. The Software Designer verifies the updated information.	
	8. The system shows a confirmation message.

Extensions

4a. The new project name conflicts with an existing project:

- 4a1. The system displays a naming conflict error.
- 4a2. The Software Designer enters a unique name.

6a. Database update fails:

- 6a1. The system displays an error and retains the original data.
- 6a2. The Software Designer retries after connectivity is restored.

Use Case 10: Retrieve Existing Project

Use Case Name	Retrieve Existing Project
Scope	UMLytics System
Level	User-Goal
Primary Actor	Software Designer
Stakeholders & Interests	• Software Designer: Wants to access previously saved projects to continue work. • Database System: Must reliably retrieve project data with all associated diagrams and history.
Preconditions	• At least one saved project exists in the database. • The database connection is active.
Postconditions	• The selected project is fully loaded into the workspace. • All diagrams, chat history, and evaluation results are available.
Main Success Scenario	
Actor Action	System Response
1. The Software Designer selects "Open Project" from the main menu.	
	2. The system queries the database and displays a list of saved projects with metadata (name, date, diagram count).
3. The Software Designer selects a project from the list.	
	4. The system retrieves all project data including diagrams, chat history, and evaluations.
5. The Software Designer views the loaded project workspace.	
	6. The system renders the project dashboard with all associated content.

Extensions

2a. No saved projects exist:

2a1. The system displays a message indicating no projects are available.

2a2. The system offers to create a new project.

4a. Database retrieval fails:

4a1. The system displays a connection error.

4a2. The Software Designer retries.

Use Case 11: Provide Voice-Based Input

Use Case Name	Provide Voice-Based Input
Scope	UMLytics System
Level	User-Goal
Primary Actor	Software Designer
Stakeholders & Interests	• Software Designer: Wants an alternative input method for accessibility or convenience. • Speech-to-Text Service: Converts spoken words into text for system processing.
Preconditions	• A microphone is connected and functional. • The Speech-to-Text service is available. • The Software Designer is in a context that accepts text input (e.g., chat, diagram description).
Postconditions	• Voice input is converted to text and placed in the active input field. • The converted text is ready for processing by the target feature.
Main Success Scenario	
Actor Action	System Response
1. The Software Designer activates the voice input button in the active panel.	
	2. The system activates the microphone and displays a recording indicator.
3. The Software Designer speaks the desired input clearly.	
	4. The system captures the audio stream and sends it to the Speech-to-Text service.
5. The Software Designer stops recording.	
	6. The Speech-to-Text service converts the audio to text and returns the transcription.
7. The Software Designer reviews the transcribed text in the input field.	
	8. The system populates the active input field with the transcribed text for further action.

Extensions

4a. No microphone is detected:

4a1. The system displays a hardware error message.

4a2. The Software Designer connects a microphone and retries.

6a. The speech is not recognized or the transcription is inaccurate:

6a1. The system displays the partial transcription.

6a2. The Software Designer manually corrects the text or re-records.

6b. Speech-to-Text service is unavailable:

6b1. The system notifies the user of the service outage.

6b2. The Software Designer uses manual text input instead.

Use Case 12: Export UML Diagram for External Use

Use Case Name	Export UML Diagram for External Use
Scope	UMLytics System
Level	User-Goal
Primary Actor	Software Designer
Stakeholders & Interests	Software Designer: Wants to export diagrams in standard formats for documentation, sharing, or presentation.
Preconditions	- A UML diagram exists in the current project. - The diagram is in a valid, renderable state.
Postconditions	- The diagram is exported to the selected file format. - The exported file is saved to the user-specified location.
Main Success Scenario	
Actor Action	System Response
1. The Software Designer selects a diagram and clicks "Export".	
	2. The system displays export options (format: PNG, PDF, SVG; resolution; destination).
3. The Software Designer selects the desired format and export settings.	
	4. The system validates the export configuration.
5. The Software Designer chooses the save location and confirms.	
	6. The system generates the diagram file in the selected format.
7. The Software Designer accesses the exported file.	
	8. The system saves the file and displays a success confirmation with the file path.
Extensions	4a. The selected format is not supported for the diagram type:

4a1. The system displays a format compatibility warning.

4a2. The Software Designer selects an alternative format.

6a. Export generation fails (e.g., rendering error):

6a1. The system displays an error message.

6a2. The Software Designer retries or selects a different format.